

BookMySeat - Full-Stack System Design & Interview Guide

Comprehensive Technical Reference, Architectural Analysis & Q&A; Handbook

Project: BookMySeat (Movie Ticket Booking System)	Target Role: Full-Stack / Backend Engineer
Framework: Django 3.2+ / Python 3.12	Database: PostgreSQL (Render) / SQLite
Deployment: Vercel (Web API) & Render (Cloud DB)	Admin Credentials: Himanshu / Himanshu@1

Executive Project Summary

BookMySeat is an enterprise-grade full-stack movie reservation platform designed to handle high-concurrency seat booking scenarios, real-time availability updates, server-verified payment gateways, high-scale business analytics, personalized recommendation engines, automated PDF ticket generation with QR code verification, and Celery asynchronous background email dispatch with auto-retries.

1. Smart Seat Reservation & Concurrency Control

Core Challenge: Multiple users selecting the exact same seats simultaneously must never result in duplicate bookings or inconsistent database states.

Technical Implementation Details:

- **Django Atomic Transactions (`transaction.atomic()`):** Enforces ACID properties during seat selection and reservation checks.
- **Row-Level Locking (`select_for_update()`):** Locks targeted seat records in PostgreSQL/SQLite during validation to prevent race conditions across parallel HTTP request threads.
- **2-Minute Lock Expiry Mechanics:** When a user selects seats, `reserved_until = timezone.now() + timedelta(minutes=2)` is set. If payment is not completed within 120 seconds, seats automatically revert to available.
- **Lazy Auto-Release Pattern:** Auto-release is performed lazily during seat status queries (`auto_release_expired_seats()`) and seat reservation attempts, eliminating the overhead of running background worker daemons.

2. Server-Side Payment Workflow & Idempotency

Core Challenge: Ticket orders must only be confirmed after valid HMAC SHA256 signature verification on the server. Network retries or duplicate webhook payloads must never create duplicate tickets or charges.

Payment Lifecycle Steps:

1. **Order Creation (`create_payment_order`):** Server generates a unique `order_id`, calculates total amount based on seat count × price, and creates a `PaymentTransaction` record with status `PENDING`.
2. **Client-Side Checkout Modal:** Razorpay / Stripe checkout modal opens with order details. User completes payment.
3. **Server Verification (`verify_payment`):** Client posts `order_id`, `payment_id`, and signature. Server re-computes HMAC SHA256 digest using secret key. On match, `PaymentTransaction` status transitions to `SUCCESS` and Booking rows are created.

4. **Strict Idempotency Protection:** If duplicate verification triggers or webhook retries hit the endpoint, the view checks if `txn.status == 'SUCCESS'` and returns the existing booking details cleanly without duplicating records.
5. **Failure & Cancellation Auto-Release (handle_payment_failure):** If payment fails or is cancelled, seats are immediately released back to Available state and transaction status becomes FAILED or CANCELLED.

3. Admin Analytics Dashboard & Query Optimization

Core Challenge: Generating complex multi-dimensional business metrics across 100,000+ bookings in sub-second response times without overloading server memory.

Key Optimization Strategies:

- **100% Database-Level ORM Aggregations:** Used Django Sum, Count, Avg, ExtractHour, and TruncDay. Zero raw model instances are loaded into Python memory.
- **Strategic B-Tree Indexing (Migration 0006 & 0007):** Added composite B-Tree database indexes on critical query paths (Booking.booked_at, PaymentTransaction.created_at, status, Seat.is_booked, Theater.time).
- **Benchmark Execution Results:** Evaluated on 100,000+ dataset using benchmark_analytics.py. Query pipeline executed in **~93.9 ms (0.0939 seconds)**.
- **CSV Report Export:** Streamlined CSV generation endpoint `/movies/admin-dashboard/export-csv/` protected by staff administrator authorization (`is_staff=True`).

4. Movie Discovery, Search & Recommendation Engine

Core Challenge: Provide multi-attribute search and filtering, dynamic match counting, paginated browsing, and personalized movie recommendations.

Engine Architecture & Components:

- **Multi-Attribute Filtering:** Title search (`Q(name__icontains=query)`), Genre, Language, City Location (`theaters__location`), Theater, Release Date, Min Rating, and Show Timings (Morning, Afternoon, Evening, Night via `theaters__time__hour`).
- **Sorting Mechanics:** Supports sorting by Newest Releases, Popularity (annotated booking count), Rating, and Ticket Price (Min ticket price across theater shows via `Min('theaters__ticket_price')`).
- **Personalized Recommendations Algorithm (get_personalized_recommendations):** Analyzes user booking history (Booking) AND recently viewed movies (RecentlyViewedMovie model + session fallback) to score and return top candidate movies matching preferred genres & languages.

5. Automated PDF Ticket Generation, QR Code & Celery Emailing

Core Challenge: Generate official PDF tickets with embedded QR code verification and dispatch them via email asynchronously without blocking checkout HTTP requests.

Architecture & Features:

- **PDF Ticket Engine (ticket_generator.py):** Generates branded PDF ticket with movie poster info, theater screen, showtime, seat numbers, payment reference, and embedded QR code (`qrcode`).
- **Asynchronous Celery Tasks (tasks.py):** Registered `send_ticket_email_task` with `@shared_task(bind=True, max_retries=3)`. Automatically attaches PDF ticket and retries up to 3 times on failure.
- **Non-Blocking Checkout & Thread Fallback:** `dispatch_ticket_email_async` triggers background execution. If Celery worker/broker is offline, it executes fallback dispatch so HTTP response is never blocked.

- **Booking History Download Endpoint:** Dedicated view /movies/transaction//ticket/ allows users to download PDF tickets from their profile page anytime.

6. Top Technical Interview Questions & Answers

Use these curated questions to ace technical system design and full-stack coding interviews.

Q1: How do you handle race conditions when two users attempt to reserve the same seat concurrently?

We use Django's database transaction blocks `transaction.atomic()` combined with row-level pessimistic locking `Seat.objects.select_for_update()`. When a user requests seat reservations, the target seat records are locked in PostgreSQL/SQLite until the atomic block finishes. If a concurrent request attempts to access the locked rows, it blocks until lock release and evaluates updated status, ensuring that seat conflicts return an immediate 409 Conflict status without duplicate bookings.

Q2: How do you enforce the 2-minute temporary seat reservation lock without background worker overhead?

Instead of maintaining continuous background worker threads or cron daemons to clear locks, we implement a lazy auto-release pattern. When a user requests seat status or attempts to reserve seats, `auto_release_expired_seats(theater_id)` executes a single atomic SQL update query: `Seat.objects.filter(reserved_until__lte=now, is_booked=False).update(reserved_by=None, reserved_until=None)`. This clears expired locks instantly at zero background server cost.

Q3: How is server-side payment security and idempotency enforced?

Bookings are never created directly from client payment callbacks. Instead, the server generates a pre-booking order ID. After payment completion, the server re-computes the HMAC SHA256 signature using the secret key and compares it to the gateway signature. To enforce idempotency, duplicate verification requests or webhook retries check `PaymentTransaction.status`. If already SUCCESS, the view returns the existing booking without creating duplicate database rows or charging the user twice.

Q4: How did you optimize ORM queries for the Admin Dashboard to achieve sub-second response times on 100,000+ records?

We eliminated Python in-memory record processing entirely by pushing all aggregations down to the database engine using Django's `.aggregate()` and `.annotate()` methods with Sum, Count, Avg, ExtractHour, and TruncDay. Additionally, we added B-Tree database indexes on `booked_at`, `created_at`, `status`, and `(theater, is_booked)`, reducing query time from seconds to ~93 ms.

Q5: How does the personalized recommendation algorithm work?

The recommendation engine (`get_personalized_recommendations`) extracts preferred genre IDs and language IDs from two sources: (1) movies the user has previously booked (Booking), and (2) movies the user recently viewed (RecentlyViewedMovie model for logged-in users or session list for guests). It queries candidate movies matching those preferred genres/languages, excludes already booked movies, orders them by rating and release date, and falls back to top-trending movies if history is sparse.

Q6: How is PDF ticket generation and asynchronous email dispatch implemented?

We generate official PDF tickets containing movie details, showtime, seat numbers, payment reference, and an embedded QR code using ReportLab and qrcode. Upon payment verification, `dispatch_ticket_email_async` delegates email dispatch to a Celery asynchronous task (`send_ticket_email_task` with `max_retries=3`). This sends emails in the background with PDF attachments without slowing down the HTTP checkout response.

Q7: How do database B-Tree indexes improve query performance in PostgreSQL?

B-Tree indexes organize column values in a balanced tree structure. Without an index, filtering by `booked_at` or `status` requires a full table scan ($O(N)$ complexity). With B-Tree indexes, database lookups navigate the tree in $O(\log N)$ time, skipping non-matching rows and executing instant range queries.

Q8: How do you handle failure and cancellation scenarios during ticket checkout?

If a payment fails or is cancelled by the user during checkout, the client triggers the `handle_payment_failure` API view. The server marks the `PaymentTransaction` status as `FAILED` or `CANCELLED`, records the failure reason, and immediately clears the seat's `reserved_by` and `reserved_until` fields, making the seats available to other users instantly.

Q9: How do you preserve filter query parameters across paginated movie discovery pages?

In `movie_list`, we clone `request.GET`, remove the 'page' parameter, and encode the remaining filters into a `querystring` context variable (e.g. `genre=action&city=Delhi`). Pagination links in the HTML template append `?{{ querystring }}&page={{ i }}`, preserving all active filters seamlessly during pagination.

Q10: Explain your dual database setup across local development and cloud production deployment.

In local development, the application defaults to lightweight SQLite. On deployment environments (e.g., Vercel + Render PostgreSQL), `settings.py` inspects environment variables (`VERCEL`, `DATABASE_URL`) and dynamically parses the PostgreSQL connection URL using `django_database_url` with connection pooling (`conn_max_age=600`).